

КУРС «ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ» · 1 СЕМЕСТР

05

Лекция 5
Классы: поля, методы, инкапсуляция

struct vs class · поля и методы · модификаторы доступа · this и fluent interface · const-методы и mutable · статические члены · friend · вложенные классы и enum class · выравнивание и padding · инкапсуляция и PIMPL

Вспоминаем, прошлую лекцию

```cpp
#include <iostream>

typedef int (*MathOp)(int, int);
int add(int a, int b) { return a + b; }
int multiply(int a, int b) { return a * b; }

int main() {
    MathOp op1 = add(10, 20);

    MathOp op2;

    op2(5, 5);

    return 0;
}
```



ПЛАН ЛЕКЦИИ

Десять глав сегодняшней лекции

01 struct vs class — единственное отличие

02 Поля и методы: внутри и вне класса

03 Модификаторы доступа

04 Указатель this и fluent interface

05 const-методы и mutable

> **Сквозная идея:** класс — это и данные (поля), и поведение (методы), и контракт доступа (инкапсуляция). Всё это — про одну конструкцию языка: пользовательский тип.

06 Статические поля и методы

07 Дружественные функции и классы

08 Вложенные классы и enum class

09 sizeof, выравнивание и padding

10 Инкапсуляция и PIMPL

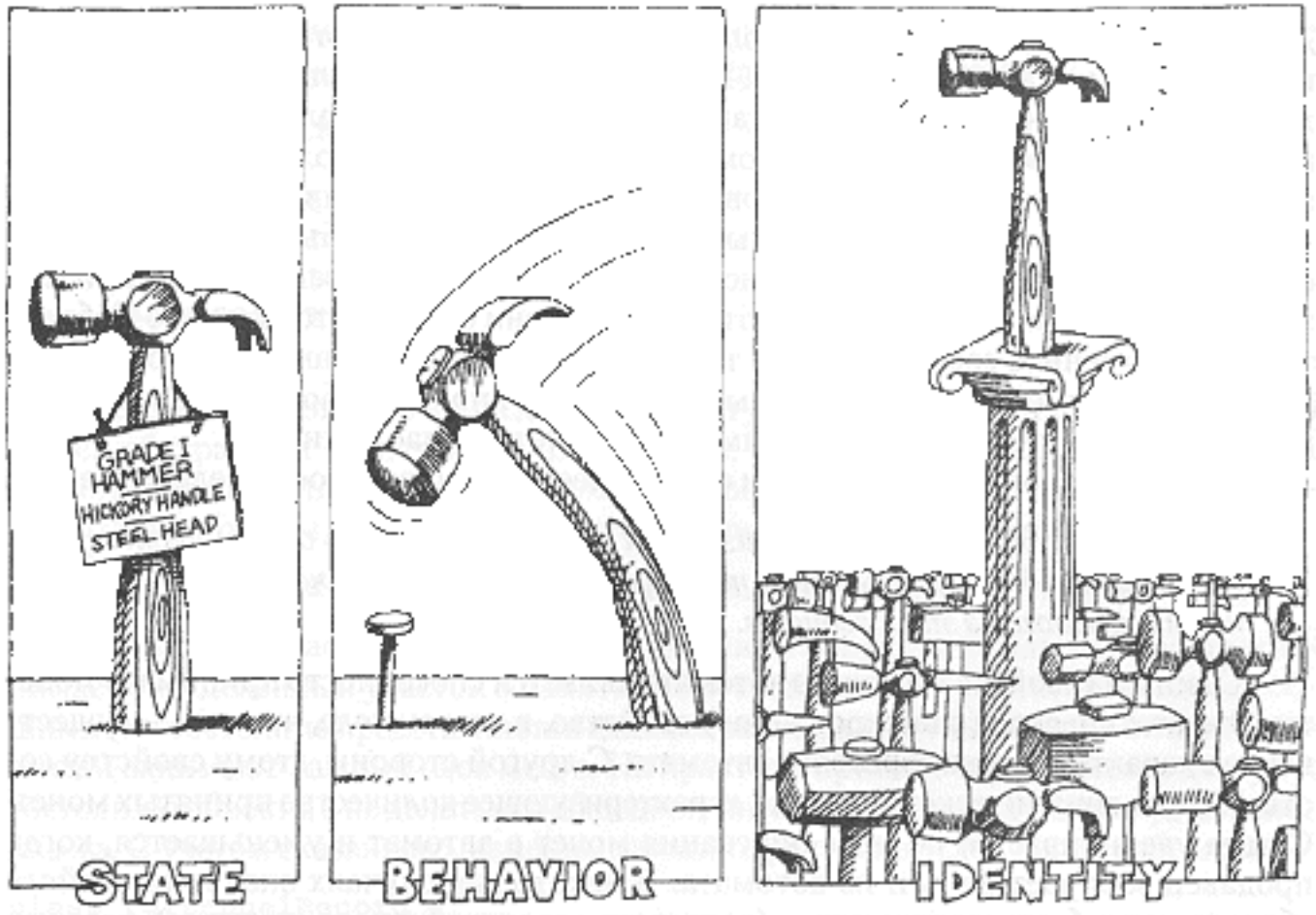
Объект

«Объект представляет собой конкретный опознаваемый предмет, единицу или сущность (реальную или абстрактную), имеющую четко определенное функциональное назначение в данной предметной области»

Smith, M. and Tockey, S. 1988. An Integrated Approach to Software Requirements Definition Using Objects. Seattle, WA: Boeing Commercial Airplane Support Division, p.132.



Свойства объекта



1. **Состояние**
   в любой момент времени объект находится в каком-либо состоянии, которое можно измерить / сравнить / скопировать

2. **Поведение**
   объект может реагировать на внешние события либо меняя свое состояние, либо создавая новые события

3. **Идентификация**
   объект всегда можно отличить от другого объекта

Класс

1. **Определение.**
   Классом будем называть группу объектов, с общей структурой и поведением.

2. Смысл программы на C++ это описание классов!

3. Даже если нужен всего один объект – мы будем описывать класс.



Инкапсуляция: управление доступом в C++

Член класса может быть **частным** (**private**), **защищенным** (**protected**) или **общим** (**public**):

1. **Частный** член класса X могут использовать только функции-члены и друзья класса X.
2. **Защищенный** член класса X могут использовать только функции-члены и друзья класса X, а так же функции-члены и друзья всех производных от X классов (рассмотрим далее).
3. **Общий** член класса можно использовать в любой функции.

Контроль доступа применяется единообразно ко всем именам. На контроль доступа не влияет, какую именно сущность обозначает имя.
Друзья класса объявляются с помощью ключевого слова **friend**. Объявление указывается в описании того класса, к частным свойствам и методам которого нужно подучать доступ.



Одно и то же, кроме доступа по умолчанию

```
struct Point {
        double x; // public по умолчанию
        double y;
};

class PointClass {
        double x; // private по умолчанию
        double y;
public:
        double getX() const { return x; }
        double getY() const { return y; }
};
```

- В C++ structи class— **одно и то же**
- Отличие — только **модификатор доступа по умолчанию**
- struct — члены по умолчанию **public** class
- — члены по умолчанию **private** У struct
- может быть private-секция

Конвенция

Простые агрегаты данных (POD) — struct. Есть инварианты и поведение — class.

ПРИМЕР

Смотрим пример: 01_struct_vs_class

```cpp
// class: члены по умолчанию private — поля скрыты, доступ только через методы.
class PointClass {
    double x;  // private по умолчанию
    double y;  // private по умолчанию
public:
    PointClass(double x_, double y_) : x(x_), y(y_) {}
    double getX() const { return x; }
    double getY() const { return y; }
};

// Доказательство, что отличие ТОЛЬКО в доступе по умолчанию:
// в struct можно явно сделать private, в class — явно public.
struct Counter {        // struct, но с инвариантом
private:                // явно закрываем поле
    int value_ = 0;
public:
    void inc() { ++value_; }
    int get() const { return value_; }
};
```

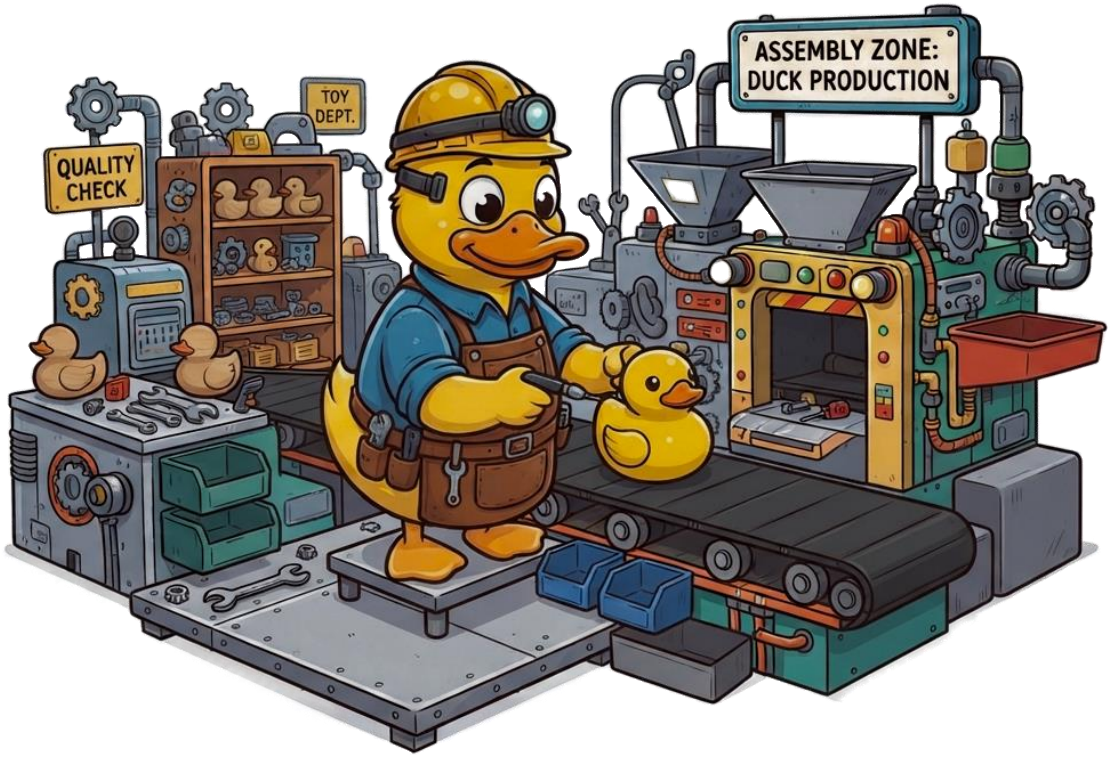


Конструктор

Если у класса есть конструктор, он вызывается всякий раз при создании объекта этого класса. Если у класса есть деструктор, он вызывается всякий раз, когда уничтожается объект этого класса.

Объект может создаваться как:

1. автоматический, который создается каждый раз, когда его описание встречается при выполнении программы, и уничтожается по выходе из блока, в котором он описан;
2. статический, который создается один раз при запуске программы и уничтожается при ее завершении;
3. объект в свободной памяти, который создается операцией new и уничтожается операцией delete;
4. объект-член, который создается в процессе создания другого класса или при создании массива, элементом которого он является.



Определение внутри класса и в .cpp

```cpp
// point.h
class Point {
        double x_, y_;
public:
        double getX() const { return x_; }
        double getY() const { return y_; }
        double distanceTo(const Point& other) const; void print() const;
};

// point.cpp
double Point::distanceTo(const Point& other) const {
        double dx = x_ - other.x_;
        double dy = y_ - other.y_;
        return std::sqrt(dx * dx + dy * dy);
}

void Point::print() const {
        std::cout << "(" << x_ << ", " << y_ << ")" << std::endl;
}
```

- Определение в теле класса — **неявно inline**
- Реализацию можно вынести в **.cpp**
- Вне класса имя квалифицируется: `Point::distanceTo`
- В заголовке — интерфейс, в .cpp— реализация

Разделяйте код

Клиенту достаточно заголовка: объявленные в `point.h`методы скрывают реализацию в `point.cpp`.

ПРИМЕР

Смотрим пример: 02_fields_methods

```
 8  class Point {
 9      double x_;
10      double y_;
11
12  public:
13      Point(double x, double y) : x_(x), y_(y) {}
14
15      // --- Определение ВНУТРИ класса (неявно inline) ---
16      // Короткие тривиальные геттеры удобно держать в заголовке.
17      double getX() const { return x_; }
18      double getY() const { return y_; }
19
20      // --- Только ОБЪЯВЛЕНИЕ; реализация в point.cpp ---
21      // Нетривиальную логику выносим в .cpp: меньше зависимостей в заголовке
22      // и быстрее перекомпиляция при изменении реализации.
23      double distanceTo(const Point& other) const;
24      void print() const;
25  };
```



public, protected, private

```
class Account {
public:
        std::string getOwner() const { return owner_; }double getBalance()
        const { return balance_; }void deposit(double amount) {
                if (amount > 0) balance_ += amount;
        }


protected:
        void setOwner(const std::string& name) { owner_ = name; }

private:
        std::string owner_;
        double balance_ = 0.0;
};
```

Модификатор	Класс	Наследник	Снаружи
public	да	да	да
protected	да	да	нет
private	да	нет	нет

> Инкапсуляция начинается здесь: private-поля + public-методы — контролируемый доступ.

ПРИМЕР

Смотрим пример: 03_access_modifiers



this и цепочки вызовов

```
class QueryBuilder {std::string table_;
        std::string condition_; int limit_ = -
        1;

public:
        QueryBuilder& from(const std::string& table) {
                this->table_ = table;
                return *this;                   // возвращаем себя
        }
        QueryBuilder& where(const std::string& cond) {condition_ = cond;
                return *this;
        }
        QueryBuilder& limit(int n) {limit_ = n;
                return *this;
        }
};



// auto q = QueryBuilder()
//          .from("users").where("age                  > 18").limit(10).build();
```

- this— неявный указатель на текущий объект
- Тип: T* const; в const-методе — const T* const return *this
- возвращает ссылку на себя
- Цепочка вызовов — **fluent interface**
- this->member— то же, что member

Использование

QueryBuilder().from("users").where("age > 18").limit(10).build()

ПРИМЕР

Смотрим пример: 04_this_fluent

```cpp
int main() {
    // Каждый метод возвращает *this, поэтому вызовы соединяются в цепочку.
    const std::string sql = sql::QueryBuilder{}
                                .from("users")
                                .where("age > 18")
                                .limit(10)
                                .build();

    std::cout << sql << '\n';
    // ожидаем: SELECT * FROM users WHERE age > 18 LIMIT 10

    // Без LIMIT и WHERE — те же методы, просто не вызываем.
    std::cout << sql::QueryBuilder{}.from("orders").build() << '\n';
    // ожидаем: SELECT * FROM orders

    return 0;
}
```



Обещание не менять — и исключения

```
class Matrix {std::vector<std::vector<double>> data_; mutable
        double cached_det_ = 0.0; mutable bool det_valid_ =
        false;

public:
        double determinant() const {
                if (!det_valid_) {
                        cached_det_ = computeDet(); // OK — mutable
                        det_valid_ = true;
                }
                return cached_det_;
        }

        void set(int i, int j, double val) {data_[i][j] = val;
                det_valid_ = false; // инвалидируем кэш
        }
};
```

- const-метод обещает **не менять состояние**
- Внутри него thisимеет тип const T*
- mutable— исключение из const
- Кэши, счётчики, мьютексы — типичные mutable
- Не-const метод **инвалидирует кэш**

Мотивация

Ленивое вычисление: determinant()считается один раз и кэшируется — вызовы const, кэш mutable.

ПРИМЕР

Смотрим пример: 05_const_mutable

```cpp
mutable double cached_det_ = 0.0;
mutable bool det_valid_ = false;

double determinant() const {                // остаётся const
    if (!det_valid_) { cached_det_ = a_*d_ - b_*c_; det_valid_ = true; }
    return cached_det_;
}
```



Члены, общие для всех объектов

```
class Logger {
        inline static int instance_count_ = 0; inline static std::string
        prefix_ = "[LOG]"; std::string name_;

public:
        Logger(const std::string& name) : name_(name) {
                ++instance_count_;
        }
        ~Logger() { --instance_count_; }

        static int getInstanceCount() { return instance_count_; }
        static void setPrefix(const std::string& p) { prefix_ = p; }

        void log(const std::string& msg) const {std::cout << prefix_ << " [" <<
                name_ << "] "
                                << msg << "\n";
        }
};
```

- Статические члены принадлежат **классу, а не объекту**
- C++17: inline static— определение прямо в заголовке
- Статический метод: **нет this**, нет доступа к нестатическим полям
- До C++17 нужно было отдельное определение в .cpp

> **До C++17:** int Logger::instance_count_ = 0;в logger.cpp— иначе ошибка множественного определения (ODR).

ПРИМЕР

Смотрим пример: 06_static_members

```cpp
class Logger {
    inline static int instance_count_ = 0;       // OK в любом числе единиц трансляции
    inline static std::string prefix_ = "[LOG]";
};
```



friend — выборочный доступ к private

```
class Vector2D {
        double x_, y_;

public:
        Vector2D(double x, double y) : x_(x), y_(y) {}

        friend double dot(const Vector2D& a, const Vector2D& b); friend std::ostream&
        operator<<(std::ostream& os,
                                                        const Vector2D& v) {
                                        return os << "(" << v.x_ << ", " << v.y_ << ")";
        }
        friend class VectorSerializer;
};

double dot(const Vector2D& a, const Vector2D& b) {
        return a.x_ * b.x_ + a.y_ * b.y_; // доступ к private
}
```

- friendдаёт доступ к private/protected
- Дружественные функция, оператор, класс
- Объявляется внутри класса
- **Не транзитивен** и **не наследуется**
- Используйте **экономно**

Важно

friendне делает функцию членом класса — лишь даёт право доступа к приватным членам.

ПРИМЕР

Смотрим пример: 07_friend

```cpp
class Vector2D {
    double x_, y_;  // private
public:
    Vector2D(double x, double y) : x_(x), y_(y) {}
    friend double dot(const Vector2D& a, const Vector2D& b);
    friend std::ostream& operator<<(std::ostream& os, const Vector2D& v);
};
```



Класс внутри класса, enum без захламления

```
class LinkedList {
public:
    enum class Error { Empty, OutOfRange, InvalidArg };
private:
    struct Node {                    // вложенный класс
        int data; Node*
        next;
        Node(int d, Node* n = nullptr) : data(d), next(n) {}
    };
    Node* head_ = nullptr; size_t
    size_ = 0;
public:
    void pushFront(int val) {
        head_ = new Node(val, head_);
        ++size_;
    }
    int front() const {
        if (!head_) throw Error::Empty;
        return head_->data;
    }
};
```

- Вложенный класс Node— деталь реализации
- enum class— scoped enum
- Не допускает **неявного** приведения к int Не
- засоряет пространство имён Обращение:
- LinkedList::Error::Empty

enum class

Нужен явный static_cast<int>(err)— никаких случайных чисел в коде.

ПРИМЕР

Смотрим пример: 08_nested_enum

```cpp
class LinkedList {
public:
    enum class Error { Empty, OutOfRange, InvalidArg };  // имена в области Error
private:
    struct Node { int data; Node* next; };               // скрыт от клиента
};

auto e = LinkedList::Error::Empty;        // явная квалификация
int n = static_cast<int>(e);              // приведение только осознанно
```



Порядок полей меняет размер

```
struct Bad {
        char a;          // 1 байт + 7 padding
        double b;        // 8 байт
        char c;          // 1 байт + 7 padding
};
// sizeof(Bad) == 24

struct Good {
        double b;        // 8 байт
        char a; char     // 1 байт
        c;               // 1 байт + 6 padding
};
// sizeof(Good) == 16

struct Empty {};
// sizeof(Empty) == 1              (уникальный адрес)
```

- Компилятор **выравнивает** поля для быстрого доступа
- Порядок полей влияет на размер объекта
- sizeof(Bad) == 24vs sizeof(Good) == 16 sizeof(Empty) == 1—
- гарантия уникальности адреса alignasи #pragma pack
- управляют выравниванием

Оптимизация

Сортируйте поля по убыванию размера: double→ char— меньше padding.

ПРИМЕР

Смотрим пример: 09_sizeof_padding

```cpp
// Плохой порядок полей: char, double, char -> много padding.
struct Bad {
    char a;    // 1 байт + 7 байт padding (чтобы double встал на адрес, кратный 8)
    double b;  // 8 байт
    char c;    // 1 байт + 7 байт padding (до размера, кратного alignof = 8)
};

// Хороший порядок: крупные поля раньше -> меньше дыр.
struct Good {
    double b;  // 8 байт
    char a;    // 1 байт
    char c;    // 1 байт + 6 байт padding
};

// Пустой класс всё равно занимает >= 1 байт: гарантия уникальности адреса.
struct Empty {};

// Управление выравниванием: alignas задаёт более строгое выравнивание.
struct alignas(16) AlignedData {
    float values[4];  // 16 байт, выровнено на границу 16
};
```



Скрыть реализацию за интерфейсом

```
// widget.h class
Widget {public:
        Widget();
        ~Widget(); Widget(Widget&&)
        noexcept;
        Widget& operator=(Widget&&) noexcept; void
        doWork();
        int getResult() const;
private:
        struct Impl;                              // forward declaration
        std::unique_ptr<Impl> pImpl_;
};
// widget.cpp
struct Widget::Impl {
        int counter = 0; std::string
        internal_data;
        void compute() { ++counter; }
};
Widget::Widget() : pImpl_(std::make_unique<Impl>()) {} Widget::~Widget() =
default;                                          // здесь Impl полон void
Widget::doWork() { pImpl_->compute(); }
```

- **Инкапсуляция** — сокрытие реализации за стабильным интерфейсом
- Контроль инвариантов · свобода менять реализацию · меньше связности
- **PIMPL** — Pointer to IMPLementation
- Ускорение компиляции · стабильный ABI
- Деструктор — в .cpp, где Impl полон

PIMPL

Изменение Impl не требует перекомпиляции клиентов: заголовок остаётся стабильным.

ПРИМЕР

Смотрим пример: 10_encapsulation_pimpl

```cpp
// PIMPL (Pointer to IMPLementation): заголовок не раскрывает детали реализации.
// Клиенту виден только указатель на неполный тип Impl — менять Impl можно,
// не перекомпилируя клиентов и не нарушая ABI.
class Widget {
public:
    Widget();
    ~Widget();  // объявлен здесь, ОПРЕДЕЛЁН в .cpp, где Impl уже полон

    // Перемещение поддерживаем, копирование по умолчанию запрещено
    // (unique_ptr некопируем). Move-операции тоже определяем в .cpp.
    Widget(Widget&&) noexcept;      You, last month • First  commit …
    Widget& operator=(Widget&&) = delete;

    void doWork();
    int getResult() const;

private:
    struct Impl;                // forward declaration: тело скрыто в .cpp
    Impl* pImpl_;  // указатель на реализацию
};
```



ИДЕЯ

Лабораторная работа №2: Класс, который сам отвечает за свою память

Вы реализуете класс с **ручным управлением памятью** — RAII, правило пяти, copy-and-swap и перегрузку операторов. Класс выбирается по семейству, алгоритм — по варианту.

✓

Один класс

RAII и правило пяти: конструкторы, деструктор и присваивание сами выделяют и освобождают память.

✓

Операторы

Перегрузка операторов: +, -, *, [], ==, <<— класс ведёт себя как встроенный тип.

✓

Семейства

String, Polynomial, Matrix. Цели и обязательная база — общие, алгоритм варианта — только ваш.

Один проект: **базу класса и операторы** пишете сами, **алгоритм варианта** покрываете тестами, утечки проверяете санитайзером.

ИТОГИ

Ключевые концепции лекции

- **struct** vs **class** — отличие только в доступе по умолчанию.
- **this** — неявный указатель на текущий объект.
- **const-метод** не модифицирует наблюдаемое состояние.
- **mutable** — исключение из const для кэшей и счётчиков.
- **inline static** — статическое поле в заголовке (C++17).
- **friend** — выборочный доступ к private, не наследуется.
- **Padding** — пустые байты для выравнивания; порядок полей важен.
- **PIMPL** — сокрытие реализации через указатель на Impl.